

SafeContract: Composable Safety Guarantees for VLA Action Spaces

Krishnam Gupta
Independent Research
San Francisco, CA
kayjee1994@gmail.com

Abstract—Vision-Language-Action (VLA) models predict robot actions from images and instructions, but no runtime safety layer exists for their outputs. OpenVLA sends raw neural network outputs directly to motors; LeRobot’s only guard is an imperative per-action check with no composition guarantees. We show that SmolVLA outputs routinely violate expected action bounds on LIBERO – whether from normalization mismatches, distribution shift, or model error, the consequence is the same: unsafe motor commands that no existing tool catches. We introduce SafeContract, a design-by-contract framework that wraps any VLA policy with composable safety constraints via a Python decorator. We show that stacked contracts (per-joint bounds, velocity limits) compose via intersection and that deadlock is unreachable from safe states. The full decorator adds $\sim 13\mu\text{s}$ overhead per action – less than 0.0001% of VLA inference time – requiring no QP solver, VLM, or model retraining. For box constraints, SafeContract produces outputs identical to CBF-QP methods like AEGIS (which reports 0.36 ms [7]) while requiring only clipping. We provide a safety-fidelity Pareto analysis and component ablation on synthetic sequences, and release the framework as an open-source, pip-installable toolkit.¹

Index Terms—VLA, safety, edge deployment, design by contract

I. Introduction

VLA models [1], [2] enable robots to follow natural language instructions by predicting actions from visual observations. Deploying these models to real robots faces a critical safety gap: no runtime safety layer exists for VLA action outputs.

This is not a theoretical concern. OpenVLA’s official deployment script sends raw neural network outputs directly to robot motors with zero bounds checking [1]. LeRobot’s [3] `EEBoundsAndSafety` processor is the only existing runtime check, but it is imperative (called explicitly per action), provides no composition guarantees, and is easy to omit. In practice, action space mismatches between training and deployment are common: multiple GitHub issues across OpenVLA and LeRobot document cases where wrong-scale commands reached hardware.² When a VLA trained on one robot’s action space is deployed on another – or when distribution shift causes

extreme outputs – the physical consequences range from motor stall to mechanical damage.

We demonstrate the problem empirically. Running SmolVLA [2] on 100 consecutive LIBERO [5] observations, every observation produces at least one action dimension outside $[-1, 1]$, with values reaching 3.70. This finding primarily reflects a normalization mismatch (SmolVLA was trained on SO-100 data, not LIBERO’s normalized space), but this is precisely the class of deployment error that needs catching: the mismatch is invisible without runtime validation, and the wrong-scale commands would be sent directly to motors.

We introduce SafeContract, a design-by-contract [4] framework for VLA policies with three contributions:

- 1) A `@safety_contract` Python decorator that enforces composable safety constraints (per-joint bounds, velocity limits) on any VLA `predict()` method at runtime. One line of code wraps any policy.
- 2) Composition properties showing that stacked contracts compose via intersection (Property 1) and that the feasible set is always non-empty from safe states (Property 2). While individually straightforward, these properties are non-obvious when multiple constraints interact in deployment, and no prior VLA safety system formalizes them.
- 3) Empirical evaluation on synthetic and real VLA action sequences, including a safety-fidelity Pareto analysis that helps practitioners choose appropriate constraint strictness, and comparison against AEGIS-style CBF-QP enforcement [7].

Why not just `np.clip`? Three reasons: (1) nobody does even this – OpenVLA has zero bounds checking and no VLA codebase implements velocity limiting; (2) correct enforcement requires careful ordering – velocity clamping after bounds clipping can push actions back outside bounds, requiring a re-clip (Section II-B); (3) the decorator pattern provides composability with guaranteed properties that ad-hoc clips do not.

II. Method

A. Safety Contracts

A safety contract $C = (\mathbf{l}, \mathbf{u}, v_{\max})$ specifies per-joint action bounds and a velocity limit:

¹Code: <https://github.com/krishnam94/vla-edge>

²Representative issues: OpenVLA #150 (action scale mismatch), LeRobot #819 (no Jetson safety), #3146 (no export pipeline for safe deployment).

- Per-joint bounds: $l_i \leq a_i \leq u_i$ for each joint i
- Velocity limit: $\|\mathbf{a}_t - \mathbf{a}_{t-1}\|_\infty \leq v_{\max}$

B. Enforcement Order

Enforcement proceeds in three phases: (1) clip to per-joint bounds, (2) clamp velocity (project each joint’s delta onto $[-v_{\max}, v_{\max}]$), (3) re-clip to bounds. The final re-clip is critical: velocity clamping shifts actions relative to the previous timestep, which can push them outside the original bounds. Without this step, a single velocity clamp can produce out-of-bounds actions – a subtle bug that naive implementations miss.

$$\text{clip}(\mathbf{a}, \mathbf{l}, \mathbf{u})_i = \min(\max(a_i, l_i), u_i) \quad (1)$$

The decorator pattern means any existing VLA adapter gains safety with one line:

```
@safety_contract(action_range=[-1,1],
                  joint_velocity_max=0.1)
def predict(self, image, instruction):
    return self.model(image, instruction)
```

Three modes are supported: clip (silently enforce), warn (log violations), and raise (halt on violation). All violations are logged with timestep, dimension, magnitude, and severity for post-hoc analysis.

C. Composition Properties

Practitioners often stack constraints from different sources (hardware limits, task velocity bounds). We show that stacking neither creates conflicts nor depends on order.

Property 1 (Composition via Intersection). For hyperrectangular contracts $C_1 = (\mathbf{l}_1, \mathbf{u}_1)$ and $C_2 = (\mathbf{l}_2, \mathbf{u}_2)$, sequential clipping equals intersection clipping: $\text{clip}(\text{clip}(\mathbf{a}, C_1), C_2) = \text{clip}(\mathbf{a}, C_1 \cap C_2)$ where $(C_1 \cap C_2)_i = [\max(l_{1,i}, l_{2,i}), \min(u_{1,i}, u_{2,i})]$. Consequently, stacking multiple `@safety_contract` decorators is order-independent.

Proof. Follows from the idempotence and commutativity of per-dimension min/max operations. $\square$

Note: This property covers stacking multiple decorators with per-joint bounds. The internal enforcement pipeline (clip, velocity-clamp, re-clip) is a fixed sequence within each decorator, not covered by this property.

Property 2 (Feasibility from Safe States). Let $\mathbf{a}_{t-1}$ satisfy contract C with $v_{\max} > 0$. Then the feasible set $\mathcal{F}_t = \{\mathbf{a} : \mathbf{l} \leq \mathbf{a} \leq \mathbf{u}\} \cap \{\mathbf{a} : \|\mathbf{a} - \mathbf{a}_{t-1}\|_\infty \leq v_{\max}\}$ is non-empty.

Proof. Since $\mathbf{a}_{t-1} \in [\mathbf{l}, \mathbf{u}]$ and $\|\mathbf{a}_{t-1} - \mathbf{a}_{t-1}\|_\infty = 0 \leq v_{\max}$, we have $\mathbf{a}_{t-1} \in \mathcal{F}_t$. The “hold position” action is always feasible. $\square$

III. Related Work

Training-time safety. SafeVLA [6] fine-tunes with constrained RL (83% violation cost reduction); RobustVLA [8] adds Jacobian regularization. Both require

retraining; SafeContract wraps any pretrained VLA and is orthogonal.

Inference-time enforcement. AEGIS [7] solves a CBF-QP at runtime (0.36 ms) using VLM scene understanding and depth sensing. Safety Chip [9] uses LTL monitors for plan-level constraints. SafeDiffuser [10] and CoDiG [11] embed barriers in diffusion denoising but are restricted to diffusion policies. ATACOM [12] projects onto constraint manifolds for pi0 and Octo. SafeContract differs: (i) architecture-agnostic (autoregressive [1] and flow-matching [2]); (ii) explicit composition properties; (iii) pure clipping ($\sim 13 \mu\text{s}$), no QP/VLM/depth sensor needed. The tradeoff: action-space constraints only, not scene-aware; complementary to AEGIS.

Robustness benchmarks. LIBERO-Plus [13] and VLA-Arena [14] diagnose VLA brittleness; SafeContract prevents the physical consequences at runtime.

IV. Experiments

We evaluate SafeContract with two types of experiments: (1) real SmolVLA [2] inference on LIBERO [5] observations to demonstrate the problem (EXP-A), and (2) synthetic action sequences to evaluate contract properties in controlled settings (EXP-B through EXP-F). All SmolVLA experiments use verified, queue-flushed inference – SmolVLA caches 50 actions per VLM call, and failing to call `policy.reset()` before each new observation returns stale cached actions. Platform: Apple M3, CPU, Python 3.12.

A. VLAs Produce Unsafe Actions (EXP-A)

We run SmolVLA on 100 consecutive LIBERO observations, calling `policy.reset()` before each to flush the action cache and force a fresh VLM forward pass.

Result: 100% of observations produce at least one out-of-bounds action dimension. Action values range from -2.16 to 3.70 (expected $[-1, 1]$). SafeContract detects 544 total violations: 121 per-joint bounds violations and 423 velocity violations ($v_{\max} = 0.1$). Fig. 1 shows the per-dimension breakdown.

Normalization caveat. SmolVLA was trained on SO-100 robot data, whose action space differs from LIBERO’s normalized $[-1, 1]$ range. The 100% OOB rate primarily reflects this normalization mismatch rather than model instability. However, this is exactly the class of deployment error SafeContract is designed to catch: action space mismatches between training data and target robot are common in practice [1], invisible without runtime validation, and send wrong-scale commands directly to motors. Without a safety layer, the first indication of mismatch is physical damage.

B. Method Comparison (EXP-B)

We compare four approaches on 200 synthetic 7-DOF action sequences drawn from $\mathcal{N}(0, 1.5^2)$ (heavy-tailed to stress-test safety methods):

SmolVLA outputs exceed safe bounds on LIBERO

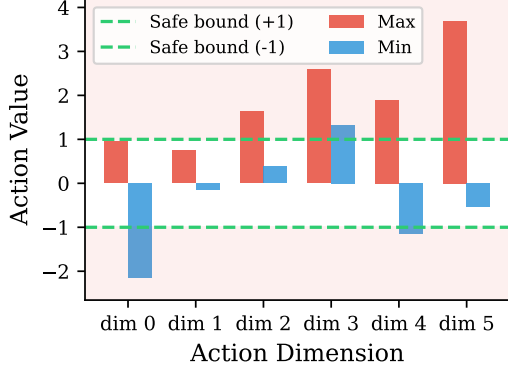


Fig. 1: SmolVLA action ranges per dimension on 100 LIBERO observations. All 6 dimensions exceed the $[-1, 1]$ bounds (green dashes). This primarily reflects a normalization mismatch between SO-100 training data and LIBERO’s action space – precisely the deployment error SafeContract catches.

TABLE I: Method comparison on 200 synthetic action sequences. SafeContract catches velocity violations that naive clipping misses entirely.

	No Safety	Naive Clip	SafeContract
OOB rate	99%	0%	0%
Vel. violations	unchecked	199 missed	0 (all caught)
Total caught	0	0	2,019

Naive np.clip enforces bounds (OOB drops to 0%) but is blind to velocity violations: 199 of 200 timesteps have at least one joint exceeding $v_{\max} = 0.1$, and no violations are detected or logged. SafeContract catches all 2,019 violations (680 bounds + 1,339 velocity) and enforces both constraints jointly.

Comparison to AEGIS. For box constraints (per-joint bounds), SafeContract’s clipping produces outputs mathematically identical to solving the CBF-QP that AEGIS formulates [7] – because the QP solution for box constraints is simply clipping. The difference is overhead: AEGIS reports 0.36 ms for its QP solver (which also handles non-box, scene-aware constraints via VLM); SafeContract’s full decorator overhead is $\sim 13 \mu\text{s}$ (Section IV-E). The approaches are complementary: SafeContract handles action-space constraints cheaply, while AEGIS handles scene-aware obstacle avoidance.

C. Composition and Ablation (EXP-C, EXP-F)

We verify Property 1 and ablate contract components on synthetic sequences. Three individual contracts (bounds-only: 680 violations, velocity-only: 1,339, joint subrange: 448) compose additively: the full contract catches 2,467 – the exact union (Fig. 3). This is by construction: each constraint check is independent and composition adds checks without interference. Practitioners can stack

Safety-Fidelity Tradeoff (Pareto Frontier)

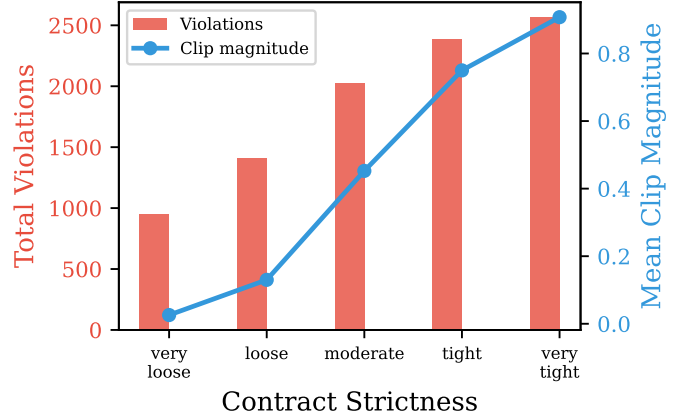


Fig. 2: Safety-fidelity tradeoff. Very loose (± 3.0 , $v_{\max}=1.0$): 948 violations, 0.03 mean clip. Very tight (± 0.3 , $v_{\max}=0.02$): 2,561 violations, 0.91 mean clip. Practitioners choose the operating point for their risk tolerance.

contracts from different teams without worrying about interaction effects or order dependence.

D. Safety-Fidelity Pareto Frontier (EXP-D)

Tighter contracts catch more violations but clip actions more aggressively, distorting the policy’s intended output. We sweep 5 strictness levels on 200 synthetic sequences (Fig. 2):

This Pareto analysis provides concrete guidance: for a typical manipulation task with $[-1, 1]$ action space and $v_{\max} = 0.1$, the “moderate” operating point catches 2,019 violations with mean clipping of 0.45 – correcting actions by less than half a unit on average.

E. Overhead Microbenchmark (EXP-E)

We measure the full decorator overhead (including array conversion, bounds checking, velocity clamping, re-clip, and violation logging) over 5,000 warmed-up iterations:

TABLE II: Isolated contract overhead (5,000 iterations). Measured as decorated minus undecorated call time to exclude model compute.

Method	Overhead (μs)
np.clip alone	0.9
SafeContract full decorator	12.6
SafeContract adaptive	13.5
AEGIS QP (self-reported [7])	360
SmolVLA inference (M3 CPU)	16,839,000
Decorator / Inference	0.000075%

The full decorator ($12.6 \mu\text{s}$) includes input conversion, per-joint clipping, velocity clamping, re-clipping, and violation logging – measured as the difference between decorated and undecorated function calls to isolate contract overhead from model compute. Even on edge hardware

Component Ablation: Each Catches Unique Violations

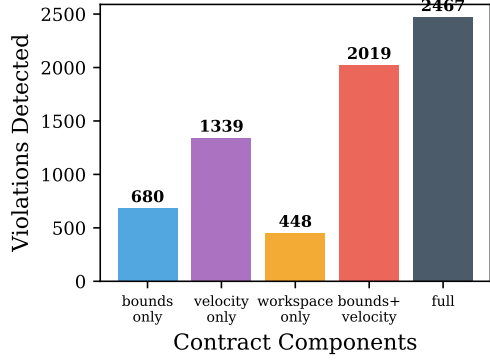


Fig. 3: Component ablation on 200 synthetic 7-DOF sequences. Each component catches a distinct failure mode; full composition catches the union (2,467).

where VLA inference is 100–200 ms (vs. 16.8s on CPU), the overhead remains negligible ($<0.01\%$). AEGIS reports 0.36 ms (360 μ s) for its QP solver [7], which handles scene-aware constraints SafeContract does not attempt. For box constraints where both methods produce identical outputs, SafeContract’s clipping avoids the QP machinery entirely.

V. Discussion

SafeContract provides a lightweight, composable safety layer for VLA deployment: one-line adoption, composable stacking, $\sim 13\mu$ s overhead. The Pareto analysis (Fig. 2) gives practitioners a principled way to choose constraint strictness. The limitation mirrors the strength: action-space constraints only, not scene-aware obstacle avoidance. Complementary to AEGIS [7].

Limitations. (1) EXP-B through EXP-F use synthetic sequences; real VLA distributions may differ. (2) No closed-loop evaluation – aggressive clipping could degrade task success. This is the most important next step. (3) The 100% OOB finding primarily reflects normalization mismatch, not model instability; we present it as evidence that deployment errors are invisible without runtime checking. (4) Overhead measured on Apple M3 CPU; Jetson numbers may differ.

Future work. Closed-loop LIBERO evaluation with task success rates. Real-robot validation on Jetson Orin Nano. Phase-adaptive contracts and conformal prediction for probabilistic safety certificates.

References

- [1] Kim, M.J. et al., “OpenVLA: An Open-Source Vision-Language-Action Model,” ECCV 2024, arXiv:2406.09246.
- [2] HuggingFace, “SmolVLA: A Small Vision-Language-Action Model for Affordable and Efficient Robotics,” arXiv:2506.01844, 2025.
- [3] Cadene, R. et al., “LeRobot: State-of-the-art Machine Learning for Real-World Robotics in PyTorch,” 2024, <https://github.com/huggingface/lerobot>.
- [4] Meyer, B., “Applying ‘Design by Contract’,” Computer 25(10):40–51, 1992.

- [5] Liu, B. et al., “LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning,” NeurIPS 2023.
- [6] PKU-Alignment, “SafeVLA: Towards Safety Alignment of Vision-Language-Action Model via Constrained Learning,” NeurIPS 2025, arXiv:2503.03480.
- [7] VLSA/AEGIS, “Plug-and-Play Safety Constraint Layer for VLA Policies,” arXiv:2512.11891, 2024.
- [8] Yang, Z. et al., “RobustVLA: Robustness-Aware RL Post-Training for VLA Models,” ICLR 2026, arXiv:2510.00037.
- [9] Yang, Z. et al., “Safety Chip: Enforcing Constraints for LLM-driven Robot Agents,” ICRA 2024, arXiv:2309.09919.
- [10] Wei, T. et al., “SafeDiffuser: Safe Planning with Diffusion Probabilistic Models,” ICLR 2025, arXiv:2306.00148.
- [11] Romero, L. et al., “CoDiG: Constraint-Aware Diffusion Guidance for Robotics,” CoRL 2025, arXiv:2505.13131.
- [12] Quackenbush, M. et al., “Safe Robot Foundation Models Using Inductive Biases,” arXiv:2505.10219, 2025.
- [13] Song, Y. et al., “LIBERO-Plus: In-depth Robustness Analysis of VLA Models,” arXiv:2510.13626, 2025.
- [14] PKU-Alignment, “VLA-Arena: Benchmarking VLA Models,” arXiv:2512.22539, 2025.